



Lớp Temperature — @property + setter validation

Cài đặt class Temperature expose 2 thuộc tính celsius và fahrenheit. Khi set celsius phải validate ≥ -273.15 — giá trị nhỏ hơn (dưới độ không tuyệt đối) thì **báo lỗi**. fahrenheit get/set quy đổi qua celsius ($F = C \cdot 9/5 + 32$).

Python: dùng @property + @celsius.setter; lỗi raise ValueError. Truy cập như attribute: t.celsius, t.fahrenheit = 100.

C++: không có @property native — viết getter/setter method: double celsius() const, void set_celsius(double), double fahrenheit() const, void set_fahrenheit(double). Báo lỗi bằng throw std::invalid_argument(...) (driver tự catch và in ERROR).

□ **Bạn chỉ nộp khai báo lớp**

File .py bạn nộp chỉ chứa **Temperature** — **không** viết main, if __name__, hay driver đọc stdin. Grader **lấy tên file bạn nộp** làm tên module, **import lớp** của bạn, **chạy driver** để ném stdin vào và so sánh stdout.

□ **Đặt tên file** là tên identifier hợp lệ (vd. Point.py, TuLanh.py, Temperature.py). Nếu đặt bậy (my-bai.py, 1.py) grader fallback student_solution.

Luồng chấm bài

Temperature.py (bạn nộp) → from Temperature import * → driver grader → stdin → stdout → so khớp = pass

□ Khung lớp bạn nộp lên

PythonC++

```
class Temperature:

    def __init__(self, celsius: float = 0.0):

        self._c = 0.0

        self.celsius = celsius # gọi setter → validate
```

```

# TODO: @property celsius – getter return self._c

# TODO: @celsius.setter – validate >= -273.15, raise ValueError nếu sai

#

# TODO: @property fahrenheit – getter return self._c * 9/5 + 32

# TODO: @fahrenheit.setter – convert và set self.celsius

```

```

#include <stdexcept>

```

```

class Temperature {

    double c_;

public:

    Temperature(double c = 0.0) { set_celsius(c); }

    double celsius() const { return c_; }

    // TODO: void set_celsius(double c) – validate c >= -273.15,

    //         throw std::invalid_argument nếu sai

    void set_celsius(double c) {

        // TODO: throw std::invalid_argument("below absolute zero");

        c_ = c;

    }

    // TODO: double fahrenheit() const { return c_ * 9.0 / 5.0 + 32.0; }

```

```
double fahrenheit() const { return 0.0; }

// TODO: void set_fahrenheit(double f) { set_celsius((f - 32) * 5/9); }

void set_fahrenheit(double f) {}

};
```

□ Grader sẽ làm gì với lớp của bạn

1. Đọc 1 số float — nhiệt độ celsius ban đầu
2. Tạo `t = Temperature(c)`; in celsius rồi in fahrenheit (Py: `t.celsius` / C++: `t.celsius()`)
3. Đặt `fahrenheit = 100` (Py: `t.fahrenheit = 100` / C++: `t.set_fahrenheit(100)`); in lại cả hai
4. Thử `set celsius = -300` — phải lỗi (Py: `ValueError` / C++: `std::invalid_argument`); driver in `ERROR`

► Xem code driver grader tự chạy

Grader chạy đoạn code sau cùng file bạn nộp. Dùng nút ở khung khai báo để xem driver cho từng ngôn ngữ:

```
from Temperature import * # file SV nộp
```

```
c0 = float(input())

t = Temperature(c0)

print(f'{t.celsius:.2f}')

print(f'{t.fahrenheit:.2f}')

t.fahrenheit = 100

print(f'{t.celsius:.2f}')

print(f'{t.fahrenheit:.2f}')

try:

    t.celsius = -300
```

```
    print('NO_ERROR')
```

```
except ValueError:
```

```
    print('ERROR')
```

```
#include <iostream>    // grader tự prepend khi compile
```

```
// ... + class bạn viết ...
```

```
int main() {
```

```
    double c0; std::cin >> c0;
```

```
    Temperature t(c0);
```

```
    std::cout << std::fixed << std::setprecision(2);
```

```
    std::cout << t.celsius() << std::endl;
```

```
    std::cout << t.fahrenheit() << std::endl;
```

```
    t.set_fahrenheit(100);
```

```
    std::cout << t.celsius() << std::endl;
```

```
    std::cout << t.fahrenheit() << std::endl;
```

```
    try {
```

```
        t.set_celsius(-300);
```

```
        std::cout << "NO_ERROR" << std::endl;
```

```
    } catch (const std::invalid_argument&) {
```

```
        std::cout << "ERROR" << std::endl;
```

```
}

    return 0;

}
```

□ Testcase mẫu — để đối chiếu format

Grader chạy *code SV + driver* rồi so sánh stdout từng dòng (strip whitespace cuối, bỏ dòng trống thừa ở cuối). Các testcase ẩn có cùng định dạng.

Ví dụ 1

stdin

```
0
```

expected stdout

```
0.00

32.00

37.78

100.00

ERROR
```

Ví dụ 2

stdin

```
100
```

expected stdout

```
100.00

212.00

37.78
```

```
100.00
```

```
ERROR
```

Ngôn ngữ hỗ trợ: bài này nhận `.py/ .cpp`. Grader tự ghép driver + class bạn nộp rồi chạy. Java sẽ mở khi có driver tương ứng.

□

Đây là bài **thực hành** — bạn **nộp lại bao nhiêu lần cũng được**. Mỗi lần nộp, hệ thống giữ điểm cao nhất. Thử, đọc error, fix, thử tiếp.